LaMAGIC2 中的 SFCI 原理详解

本文结合 LaMAGIC2 论文及我们此前的讨论，解释 SFCI 的定义、表示原理、共享线性映射的作用和适用范围。文中的电路连接例子是教学示意，用于解释编码规则，不代表论文中的原始测试样本。

一、SFCI 是什么

SFCI 是 LaMAGIC2 提出的一种电路拓扑表示方法，全称为 Succinct Float-input Canonical Formulation with Identifier。可以理解为一种简洁的、带器件标识符的规范化表示方式，并配合浮点数值输入。

它要解决的问题是：怎样把电路中的器件及其连接关系转换成语言模型能够接收和生成的序列，同时尽量减少冗余信息。SFCI 不是新的 Transformer 架构，也不是电路仿真算法；它主要改变的是电路结构的表达方式。

SFCI 包含两个需要区分的部分：

- 拓扑表示：用稀疏超边列表描述实际存在的电气连接，并保留器件类型和实例编号。
- 数值输入：用共享线性投影把浮点数规格映射到 Transformer 的向量维度，使数值不必先作为普通文本经过 Tokenizer。

二、为什么需要 SFCI

LaMAGIC 第一代方法尝试了矩阵表示和稀疏表示。不同方法各有优点，但也存在不足。

表示方法	主要特点与不足
CF	连接表示较稀疏、序列较短；但把 Sa0、Sa1 当成独立 Token 时，不利于模型共享学习它们同属 Sa 类型。
PM	用文本表示矩阵，能够保留器件类型信息；但矩阵中大量位置表示不存在的连接，产生冗余 Token。

FM	引入浮点数值输入映射；但拓扑输出仍是矩阵，序列规模仍随顶点数呈平方级增长。
SFM	进一步精简输入标签和占空比表达；但输出仍为矩阵，没有彻底解决矩阵规模问题。

SFCI 将几种方法中有用的设计结合起来：采用稀疏连接表示，保留器件类型信息，把器件类型与实例编号分开，并继续使用浮点数值输入映射。

三、理解 SFCI 的基础：电路超图

3.1 为什么普通图不够用

电路中的导线会形成电气网络（Net）。同一个 Net 可能同时连接多个器件端口，而不只是两个对象。普通图的一条边通常连接两个顶点；超图则允许一条超边同时连接多个顶点，因此更适合描述这种连接关系。

概念	**在电路中的含义**
顶点（Vertex）	表示端口或器件节点，例如 VIN、VOUT、GND、Sa、C、L。
超边（Hyperedge）	表示同一个电气网络中所有相连顶点构成的集合。

因此，顶点表示“谁参与连接”，超边表示“哪些对象属于同一个电气网络”。超边不是一只器件，也不等于某一小段物理导线。

3.2 一个简单例子

假设一个教学示意电路包含 VIN、VOUT、GND，以及两个开关 Sa0、Sa1、电容 C2 和电感 L3。它的连接关系可以抽象成四条超边：

超边	**所属顶点**
E1	{VIN, Sa0}

E2	{VOUT, C2, L3}
E3	{GND, Sa1, C2}
E4	{Sa0, Sa1, L3}

Sa0 同时出现在 E1 和 E4 中，表示它连接到两个不同的电气网络；C2 出现在 E2 和 E3 中；L3 出现在 E2 和 E4 中。通过器件在不同超边中的出现情况，可以表达它与各个网络的连接关系。

四、SFCI 怎样把超边写成 Token 序列

SFCI 不逐个列出矩阵中所有不存在的连接，而是直接把每条超边中实际存在的顶点写出来。不同超边之间用分隔符区分。以上面的教学例子为例，可以写成：

```
<VIN> <Sa> 0,
<VOUT> <C> 2 <L> 3,
<GND> <Sa> 1 <C> 2,
<Sa> 0 <Sa> 1 <L> 3
```

逗号前后是不同的超边；同一组中的顶点属于同一个电气网络。

序列片段	含义
<VIN> <Sa> 0	VIN 与 Sa 的第 0 个实例属于同一个 Net。
<VOUT> <C> 2 <L> 3	VOUT、C2 和 L3 属于同一个 Net。
<GND> <Sa> 1 <C> 2	GND、Sa1 和 C2 属于同一个 Net。
<Sa> 0 <Sa> 1 <L> 3	Sa0、Sa1 和 L3 属于同一个 Net。

所以，SFCI 的核心不是记录“器件 A 和器件 B 之间画了一条线”，而是记录“哪些器件端点或端口属于同一个电气网络”。

五、为什么要把器件类型和编号分开

假设电路里有两个开关 Sa0 和 Sa1。它们属于同一种器件类型，但又是两个不同的器件实例。

如果把它们写成两个互不相关的整体 Token（例如 <Sa0>、<Sa1>），模型就不容易共享学习它们的共同类型特征。SFCI 将其拆成类型和编号：

```
<Sa> 0
<Sa> 1
```

- <Sa> 表示器件类型，即它是 Sa 类开关。
- 0 或 1 表示实例编号，用来区分同类型的不同器件。

这样，两个器件既共享类型 Token，又保留不同的实例身份。VIN、VOUT、GND 等唯一端口不需要像器件实例一样编号。论文的消融实验显示，去掉输出中的器件类型 Token 会使性能变差，因此完整 SFCI 保留了类型信息。

六、共享线性映射：浮点数怎样进入 Transformer

SFCI 名称中的 Float-input 与数值规格输入有关。这里要把两个流程区分开：拓扑输出用超边序列表示；电压转换比、效率等浮点规格则通过共享线性投影进入模型。

6.1 普通文本输入

```
文本："Efficiency: 0.93"
        ↓
Tokenizer 将文本切分为 Token
        ↓
Token ID 经过普通 Embedding 层
        ↓
Transformer
```

普通文本路径中，0.93 被当作字符串的一部分交给 Tokenizer。它可能被切成一个 Token，也可能被切成多个 Token，具体取决于 Tokenizer 的词表。

6.2 浮点数输入

```
原始浮点数：x = 0.93
        ↓
共享线性投影：e = W x + b
        ↓
```

得到符合模型隐藏维度的向量

↓

Transformer

在这条数值输入路径中，浮点数直接通过线性层变成向量，不需要先转换成文字 Token，也不需要再经过普通文本 Tokenizer。Transformer 处理的是向量；只要浮点数被映射成与模型隐藏维度一致的向量，就可以与文字 Token 的向量表示一起输入模型。

6.3 “共享”是什么意思

共享线性映射是指：不同数值输入复用同一套可学习参数 W 和 b，而不是每种指标都单独设置一套线性层。

$$e_i = W x_i + b$$

例如，电压转换比 0.85 和效率 0.93 分别经过同一个映射函数。两个输入使用相同的 W 和 b，但因为输入数值不同，得到的向量也不同。共享的是映射参数，不是输出向量。

线性映射本身并不知道某个数值代表电压转换比还是效率。指标的含义需要由输入顺序、位置或其他上下文信息提供。线性映射只解决“数值怎样变成模型可接收的向量”，不意味着模型已经理解了数值与电路之间的关系。

七、SFCI 为什么能减少 Token

矩阵表示需要描述大量顶点对之间的关系，其中很多位置对应“不存在连接”，因此会产生较多冗余 Token。SFCI 则直接列出实际超边中包含的顶点。

SFCI 的拓扑序列长度可以近似理解为所有超边中的顶点出现次数之和：

$$L \approx \Sigma_{e \in E} |e| = \Sigma_{v \in V} d(v)$$

其中 $d(v)$ 表示顶点 v 出现在多少条超边中。在论文所研究的电源变换器表示中，端口顶点只属于一个网络，两端器件顶点最多属于两个网络，因此 $d(v) \leqslant 2$，可得到 $L \leqslant 2|V|$，输出长度为 $O(|V|)$。

这与矩阵表示通常需要 $O(|V|^2)$ 的规模不同。需要强调：SFCI 的线性长度结论依赖论文中的有界度假设，不能直接推广到任意超图。

八、SFM 与 SFCI 的区别

比较点	SFM	SFCI
浮点数输入	使用线性映射处理数值规格。	同样使用线性映射处理数值规格。
拓扑输出	仍然是矩阵表示。	改为稀疏超边序列。
器件身份	在矩阵表示中保留器件信息。	明确拆分器件类型与实例编号。
输出规模	仍为 $O(\|V\|^2)$。	在论文的有界度假设下为 $O(\|V\|)$。

简要来说，SFM 主要精简矩阵方案的输入和标签表达；SFCI 则进一步改变拓扑输出形式，从矩阵转为稀疏超边列表。

九、SFCI 能否用于 MOS 晶体管电路

论文讨论了将表示方法扩展到晶体管电路的方向。MOSFET 有 D、G、S、B 四个主要端子。如果把整只 MOS 管仅表示成一个没有引脚区分的顶点，就无法说明这四个端子分别连接到哪些 Net。

一种思路是把器件实例展开成引脚级节点，例如：

```
<NMOS> 0 D
<NMOS> 0 G
<NMOS> 0 S
<NMOS> 0 B
```

再把属于同一个 Net 的引脚节点放进同一条超边，就可以表示晶体管各引脚的连接关系。

但必须注意：论文展示了晶体管级逆变器的表示示例，并讨论了 D/G/S/B 引脚级扩展思路；这不等于已经在大规模、任意晶体管级模拟 IC 上完成充分训练和验证。

十、SFCI 的作用与局限

SFCI 能够：

- 把已知电路拓扑编码成结构清晰、相对紧凑的序列。
- 用超边表达一个电气网络同时连接多个对象的情况。
- 通过类型 Token 和实例编号，区分器件类别与具体器件。
- 在论文设定的有界度条件下减少输出 Token 数量。

SFCI 本身不能：

- 仅凭编码就自动识别 OTA、CMFB、Bandgap 等功能模块。
- 保证模型生成的拓扑一定满足电气性能要求。
- 替代 SPICE 仿真、实际芯片测量或故障诊断算法。
- 证明模型已经能够理解任意晶体管级模拟集成电路。

十一、总结

理解 SFCI，抓住三个核心即可：

1. 连接关系：把电路看成超图，顶点表示端口或器件节点，超边表示同一个电气网络中的顶点集合。
2. 器件身份：把器件类型与实例编号分开，例如 <Sa> 0 和 <Sa> 1，使模型既能识别类型，又能区分实例。
3. 数值输入：用共享线性投影把浮点规格直接映射成 Transformer 所需的向量；这条数值通道不需要经过普通文本 Tokenizer。

因此，SFCI 的本质是：用紧凑、规范的超边序列表达电路拓扑，同时配合适合浮点规格的输入方式，让语言模型更有效地学习“设计规格—电路拓扑”之间的关系。它是一种结构化表示方法，而不是电路理解能力本身。

参考文献

Chang, Chen-Chia, Wan-Hsuan Lin, Yikang Shen, Yiran Chen, and Xin Zhang.
“LaMAGIC2: Advanced Circuit Formulations for Language Model-Based Analog

Topology Generation.” Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Proceedings of Machine Learning Research, Vol. 267, pp. 7351 – 7360.

论文预印本：arXiv:2506.10235。

官方论文页面：https://proceedings.mlr.press/v267/chang25b.html

建议核查位置：第 2 节（任务与超图表示）、第 3 节（数值输入）、第 4.1 节（SFM）、第 4.2 节（SFCI）、第 6.4 节（晶体管级表示扩展）。